

# Assignment 2 Solutions

Charlie Li

May 6, 2026

## 1 Understanding word2vec

(a)

$y_w$  is 0 everywhere except for  $y_o$ , which equals 1.

(b)

1.

$$\begin{aligned}
 \frac{\partial}{\partial \mathbf{v}_c} \mathbf{J}_{\text{naive-softmax}}(\mathbf{v}_c, o, \mathbf{U}) &= \frac{\partial}{\partial \mathbf{v}_c} (-\log P(O = o \mid C = c)) \\
 &= -\frac{\partial}{\partial \mathbf{v}_c} \log \frac{\exp(\mathbf{u}_o^\top \mathbf{v}_c)}{\sum_{w \in \text{Vocab}} \exp(\mathbf{u}_w^\top \mathbf{v}_c)} \\
 &= -\frac{\sum_{w \in \text{Vocab}} \exp(\mathbf{u}_w^\top \mathbf{v}_c)}{\exp(\mathbf{u}_o^\top \mathbf{v}_c)} \times \frac{\partial}{\partial \mathbf{v}_c} \frac{\exp(\mathbf{u}_o^\top \mathbf{v}_c)}{\sum_{w \in \text{Vocab}} \exp(\mathbf{u}_w^\top \mathbf{v}_c)} \\
 \frac{\partial}{\partial \mathbf{v}_c} \frac{\exp(\mathbf{u}_o^\top \mathbf{v}_c)}{\sum_{w \in \text{Vocab}} \exp(\mathbf{u}_w^\top \mathbf{v}_c)} &= \frac{\frac{\partial}{\partial \mathbf{v}_c} \exp(\mathbf{u}_o^\top \mathbf{v}_c) \sum_{w \in \text{Vocab}} \exp(\mathbf{u}_w^\top \mathbf{v}_c) - \exp(\mathbf{u}_o^\top \mathbf{v}_c) \frac{\partial}{\partial \mathbf{v}_c} \sum_{w \in \text{Vocab}} \exp(\mathbf{u}_w^\top \mathbf{v}_c)}{\left(\sum_{w \in \text{Vocab}} \exp(\mathbf{u}_w^\top \mathbf{v}_c)\right)^2} \\
 &= \frac{\mathbf{u}_o^\top \exp(\mathbf{u}_o^\top \mathbf{v}_c) \sum_{w \in \text{Vocab}} \exp(\mathbf{u}_w^\top \mathbf{v}_c) - \exp(\mathbf{u}_o^\top \mathbf{v}_c) \sum_{w \in \text{Vocab}} \mathbf{u}_w^\top \exp(\mathbf{u}_w^\top \mathbf{v}_c)}{\left(\sum_{w \in \text{Vocab}} \exp(\mathbf{u}_w^\top \mathbf{v}_c)\right)^2} \\
 &= \frac{\mathbf{u}_o^\top \exp(\mathbf{u}_o^\top \mathbf{v}_c)}{\sum_{w \in \text{Vocab}} \exp(\mathbf{u}_w^\top \mathbf{v}_c)} - \frac{\exp(\mathbf{u}_o^\top \mathbf{v}_c) \sum_{w \in \text{Vocab}} \mathbf{u}_w^\top \exp(\mathbf{u}_w^\top \mathbf{v}_c)}{\left(\sum_{w \in \text{Vocab}} \exp(\mathbf{u}_w^\top \mathbf{v}_c)\right)^2} \\
 \frac{\partial}{\partial \mathbf{v}_c} \mathbf{J}_{\text{naive-softmax}}(\mathbf{v}_c, o, \mathbf{U}) &= -\mathbf{u}_o^\top + \frac{\sum_{w \in \text{Vocab}} \mathbf{u}_w^\top \exp(\mathbf{u}_w^\top \mathbf{v}_c)}{\sum_{w \in \text{Vocab}} \exp(\mathbf{u}_w^\top \mathbf{v}_c)} \\
 &= -\mathbf{u}_o^\top + E_o[\mathbf{u}_o^\top \mid C = c]
 \end{aligned}$$

With shape convention,

$$\frac{\partial}{\partial \mathbf{v}_c} \mathbf{J}_{\text{naive-softmax}}(\mathbf{v}_c, o, \mathbf{U}) = -\mathbf{u}_o + E_o[\mathbf{u}_o \mid C = c] = -\mathbf{U}y + \mathbf{U}\hat{y} = \mathbf{U}(-y + \hat{y})$$

2.

$$\mathbf{U}(-y + \hat{y}) = 0$$

$$-y + \hat{y} \in \text{Null}(\mathbf{U})$$

3. Adding  $\mathbf{u}_o$  to the word vector  $v_c$  makes it more aligned to  $\mathbf{u}_o$ , which increases the dot product between  $\mathbf{u}_o$  and  $\mathbf{v}_c$ , thus increasing the probability  $P(O = o \mid C = c)$ . Subtracting the expectation of  $\mathbf{u}_o$  zeros out the current guess and pushes the vector away from wrong outside words.

(c)

It would affect the classification when a word that has a stronger emotion also has a word vector with a larger magnitude. It would not affect the classification when the length of the word vector is not correlated with the strength of the meaning of the word.

(d)

$$\frac{\partial J}{\partial U} = \left[ \frac{\partial J}{\partial \mathbf{u}_1}, \frac{\partial J}{\partial \mathbf{u}_2}, \dots, \frac{\partial J}{\partial \mathbf{u}_{|\text{Vocab}|}} \right]$$

(e)

$$\begin{aligned} \frac{\partial J}{\partial \mathbf{u}_i} &= -\frac{\partial}{\partial \mathbf{u}_i} \log \frac{\exp(\mathbf{u}_o^\top \mathbf{v}_c)}{\sum_{w \in \text{Vocab}} \exp(\mathbf{u}_w^\top \mathbf{v}_c)} \\ &= -\frac{\sum_{w \in \text{Vocab}} \exp(\mathbf{u}_w^\top \mathbf{v}_c)}{\exp(\mathbf{u}_o^\top \mathbf{v}_c)} \\ &\quad \left[ \frac{\frac{\partial}{\partial \mathbf{u}_i} \exp(\mathbf{u}_o^\top \mathbf{v}_c) \sum_{w \in \text{Vocab}} \exp(\mathbf{u}_w^\top \mathbf{v}_c) - \exp(\mathbf{u}_o^\top \mathbf{v}_c) \frac{\partial}{\partial \mathbf{u}_i} (\sum_{w \in \text{Vocab}} \exp(\mathbf{u}_w^\top \mathbf{v}_c))}{(\sum_{w \in \text{Vocab}} \exp(\mathbf{u}_w^\top \mathbf{v}_c))^2} \right] \\ &= AB \end{aligned}$$

case 1:  $i \neq o$

$$\begin{aligned} B &= \frac{-\exp(\mathbf{u}_o^\top \mathbf{v}_c) \mathbf{v}_c \exp(\mathbf{u}_i^\top \mathbf{v}_c)}{(\sum_{w \in \text{Vocab}} \exp(\mathbf{u}_w^\top \mathbf{v}_c))^2} \\ AB &= \frac{\mathbf{v}_c \exp(\mathbf{u}_i^\top \mathbf{v}_c)}{\sum_{w \in \text{Vocab}} \exp(\mathbf{u}_w^\top \mathbf{v}_c)} \\ &= \mathbf{v}_c \hat{y}_i \end{aligned}$$

case 2:  $i = o$

$$\begin{aligned} B &= \frac{\mathbf{v}_c \exp(\mathbf{u}_o^\top \mathbf{v}_c)}{\sum_{w \in \text{Vocab}} \exp(\mathbf{u}_w^\top \mathbf{v}_c)} + B_{\text{case 1}} \\ AB &= -\mathbf{v}_c + \mathbf{v}_c \hat{y}_i \end{aligned}$$

unified equation

$$\frac{\partial J}{\partial \mathbf{u}_i} = \mathbf{v}_c (\hat{y}_i - y_i)$$

(f)

$$f'(x) = \begin{cases} 1 & x > 0 \\ \alpha & x < 0 \end{cases}$$

(g)

$$\begin{aligned}\sigma'(x) &= \frac{(e^x)'(e^x + 1) - e^x(e^x + 1)'}{(e^x + 1)^2} \\ &= \frac{e^x(e^x + 1) - e^x e^x}{(e^x + 1)^2} \\ &= \frac{e^x}{e^{2x} + 1 + 2e^x} \\ &= \frac{1}{e^x + e^{-x} + 2} \\ 1 + e^{-x} &= \frac{1}{\sigma(x)} \\ e^{-x} &= \frac{1 - \sigma(x)}{\sigma(x)} \\ e^x &= \frac{\sigma(x)}{1 - \sigma(x)} \\ \sigma'(x) &= \frac{1}{\frac{\sigma(x)}{1 - \sigma(x)} + \frac{1 - \sigma(x)}{\sigma(x)} + 2} \\ &= \frac{\sigma(x)(1 - \sigma(x))}{\sigma^2(x) + (1 - \sigma(x))^2 + 2\sigma(x)(1 - \sigma(x))} \\ &= \sigma(x)(1 - \sigma(x))\end{aligned}$$

## 2 Neural Networks Optimization

(a)

1. The new gradient is multiplied by a coefficient  $1 - \beta_1$ , which makes the change smaller. It accumulates the history of the gradients, which prevents the model from being trapped in shallow local minima.
2. Parameters with smaller  $v$  will get larger updates, which corresponds to smaller gradient of  $J$  with respect to that parameter. This helps with learning by helping parameters get out of a plateau or reach a minimum faster.

(b)

- 1.

$$\begin{aligned}h_{drop,i} &= \lambda dh_i \\ E[h_{drop,i}] &= E[\lambda dh_i] \\ &= \lambda E[d] h_i \\ &= h_i \\ \lambda &= \frac{1}{E[d]} \\ &= \frac{1}{p_{drop}}\end{aligned}$$

### 3 Neural Transition-Based Dependency Parsing

(a)

| Stack                                       | Buffer                               | New dependency         | Transition |
|---------------------------------------------|--------------------------------------|------------------------|------------|
| [ROOT, presented, my]                       | [findings, at, the, NLP, conference] |                        | SHIFT      |
| [ROOT, presented, my, findings]             | [at, the, NLP, conference]           |                        | SHIFT      |
| [ROOT, presented, findings]                 | [at, the, NLP, conference]           | findings → my          | LEFT-ARC   |
| [ROOT, presented]                           | [at, the, NLP, conference]           | presented → findings   | RIGHT-ARC  |
| [ROOT, presented, at]                       | [the, NLP, conference]               |                        | SHIFT      |
| [ROOT, presented, at, the]                  | [NLP, conference]                    |                        | SHIFT      |
| [ROOT, presented, at, the, NLP]             | [conference]                         |                        | SHIFT      |
| [ROOT, presented, at, the, NLP, conference] | []                                   |                        | SHIFT      |
| [ROOT, presented, at, the, conference]      | []                                   | conference → NLP       | LEFT-ARC   |
| [ROOT, presented, at, conference]           | []                                   | conference → the       | LEFT-ARC   |
| [ROOT, presented, conference]               | []                                   | conference → at        | LEFT-ARC   |
| [ROOT, presented]                           | []                                   | presented → conference | RIGHT-ARC  |
| [ROOT]                                      | []                                   | ROOT → presented       | RIGHT-ARC  |

(b)

$2n$  steps. It takes  $n$  shifts to empty the buffer, and  $n$  right or left arcs to empty the stack to one element.

(e)

i

$$\begin{aligned}
 h_i &= \text{ReLU}(\mathbf{x}\mathbf{W}_i + b_{1,i}) \text{ where } \mathbf{W}_i \text{ is column } i \text{ in matrix } \mathbf{W} \\
 \frac{\partial h_i}{\partial x_j} &= \frac{\partial \text{ReLU}(\mathbf{x}\mathbf{W}_i + b_{1,i})}{\partial \mathbf{x}\mathbf{W}_i + b_{1,i}} \frac{\partial \mathbf{x}\mathbf{W}_i + b_{1,i}}{\partial x_j} \\
 &= [(\mathbf{x}\mathbf{W}_i + b_{1,i}) > 0] W_{j,i}
 \end{aligned}$$

ii

$$\begin{aligned}
 \frac{\partial J}{\partial \hat{y}_i} &= -\frac{y_i}{\hat{y}_i} \\
 \hat{y}_i &= \frac{e^{l_i}}{\sum_{j=1}^3 e^{l_j}} \\
 \frac{\partial \hat{y}_i}{\partial l_i} &= \frac{e^{l_i}(\sum_{j=1}^3 e^{l_j}) - e^{l_i}e^{l_i}}{(\sum_{j=1}^3 e^{l_j})^2} \\
 \frac{\partial J}{\partial \hat{l}_i} &= -\frac{y_i}{\hat{y}_i} \frac{e^{l_i}(\sum_{j=1}^3 e^{l_j}) - e^{2l_i}}{(\sum_{j=1}^3 e^{l_j})^2}
 \end{aligned}$$

iii

- UAS on dev set: 72.10
- UAS on test set: 89.31